

# Formal Methods for System Design

## Chapter 4: Computation tree logic

Mickael Randour

LoFI — Logic and Foundations of Intelligent Systems

Mathematics Department, Faculty of Science, UMONS

October 2026

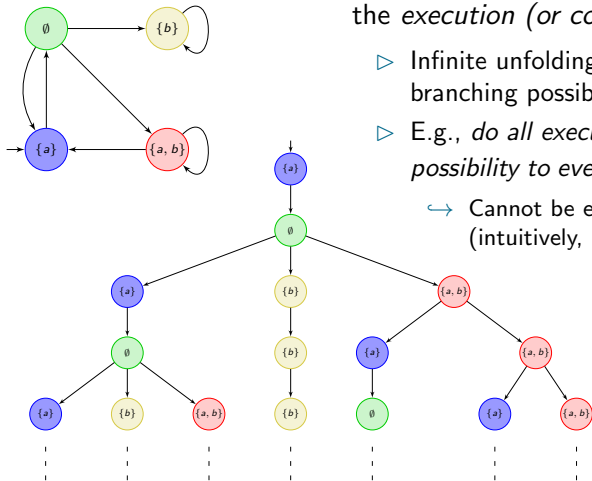


- 1 CTL: a specification language for BT properties
- 2 CTL model checking
- 3 CTL vs. LTL
- 4 CTL\*



## Branching time semantics: a reminder

- **Branching time semantics** deals with the *execution (or computation) tree*.
  - ▷ Infinite unfolding considering all branching possibilities.
  - ▷ E.g., *do all executions always have the possibility to eventually reach  $\{b\}$ ?* **Yes.**
    - ↪ Cannot be expressed as a LT property (intuitively, requires *branching*).





## Different notions of time

- ⇒ We will see that the expressiveness of LTL and CTL are incomparable. . .

...and we will sketch  $CTL^*$ , a logic which subsumes both LTL and CTL.





## CTL syntax

## Core syntax

## CTL syntax

Given the set of atomic propositions  $AP$ , CTL *state formulae* are formed according to the following grammar:

$$\Phi ::= \text{true} \mid a \mid \Phi \wedge \Psi \mid \neg \Phi \mid \exists \phi \mid \forall \phi$$

where  $a \in AP$  and  $\phi$  is a path formula. CTL *path formulae* are formed according to the following grammar:

$$\phi ::= \bigcirc \Phi \mid \Phi \cup \Psi$$

where  $\Phi$  and  $\Psi$  are state formulae.

⇒ The syntax enforces the presence of a path quantifier before every temporal operator.

→ When we just say *CTL formula*, we mean *CTL state formula*.



# CTL syntax

## Examples (2/2)

### CTL syntax reminder

$$\Phi ::= \text{true} \mid a \mid \Phi \wedge \Psi \mid \neg \Phi \mid \exists \phi \mid \forall \phi$$

$$\phi ::= \bigcirc \Phi \mid \Phi \cup \Psi$$

- Is  $\Phi = \exists((\forall \bigcirc a) \cup (a \wedge b))$  a valid CTL formula?
  - ▷ **Yes**, because  $\Psi_1 = \forall \bigcirc a$  and  $\Psi_2 = a \wedge b$  are valid state formulae, hence  $\phi = \Psi_1 \cup \Psi_2$  is a valid path formula, hence  $\Phi = \exists \phi$  is a valid state formula.
- Is  $\Phi = \exists \bigcirc (a \cup b)$  a valid CTL formula?
  - ▷ **No**, because  $\phi = a \cup b$  is a valid *path* formula whereas we require a *state* formula at this position. I.e., one needs to insert quantification for the  $\cup$  operator.

# CTL syntax

## Derived operators

Boolean operators false,  $\vee$ ,  $\oplus$ ,  $\rightarrow$ ,  $\leftrightarrow$  derived as for LTL.

Other derivations also similar:

$$\exists \Diamond \Phi \equiv \exists (\text{true} \cup \Phi) \quad \text{*potentially*}$$

$$\forall \Diamond \Phi \equiv \forall (\text{true} \cup \Phi) \quad \text{*inevitably*}$$

$$\exists \Box \Phi \equiv \neg \forall \Diamond \neg \Phi \quad \text{*potentially always*}$$

$$\forall \Box \Phi \equiv \neg \exists \Diamond \neg \Phi \quad \text{*invariantly*}$$

$$\exists (\Phi \text{ W } \Psi) \equiv \neg \forall ((\Phi \wedge \neg \Psi) \cup (\neg \Phi \wedge \neg \Psi)) \quad \text{*weak until*}$$

$$\forall (\Phi \text{ W } \Psi) \equiv \neg \exists ((\Phi \wedge \neg \Psi) \cup (\neg \Phi \wedge \neg \Psi))$$

Would  $\forall \Box \Phi \equiv \forall \neg \Diamond \neg \Phi$  be a correct derivation (similar to LTL)?

No! Because  $\neg$  cannot be applied to *path* formulae.

$\Rightarrow$  Derivations are based on the duality between  $\exists$  and  $\forall$ .

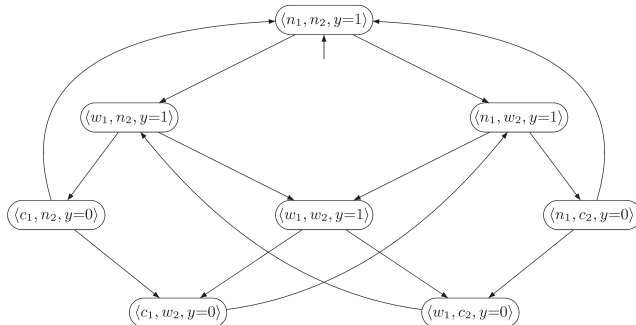
# CTL syntax

## Precedence order

Same rules as for LTL, with quantifiers  $\exists$ ,  $\forall$  directly linked to the following path formula.

# Formalizing LT/BT properties in CTL

## Safety

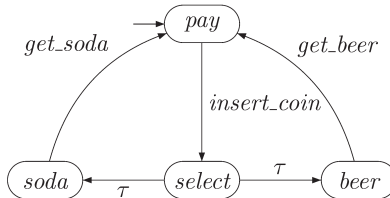


*TS for semaphore-based mutex [BK08] (Ch. 2).*

- ▷  $AP = \{crit_1, crit_2\}$ , natural labeling.
- ▷ In LTL,  $\neg \Diamond (crit_1 \wedge crit_2)$  or  $\Box (\neg crit_1 \vee \neg crit_2)$ .
- ↔ In CTL,  $\neg \exists \Diamond (crit_1 \wedge crit_2)$  or  $\forall \Box (\neg crit_1 \vee \neg crit_2)$ .

# Formalizing LT/BT properties in CTL

## Liveness



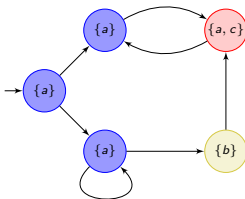
*Beverage vending machine [BK08] (Ch. 2).*

- ▷  $AP = \{paid, drink\}$ , natural labeling.
- ▷ In LTL,  $\square\diamond drink$ .
- ↪ In CTL,  $\forall\square\forall\diamond drink$ . Intuitively, for all paths, it is true at every step that all futures will eventually reach *drink*.

⇒ Formal proof after proper definition of the semantics.

# Formalizing LT/BT properties in CTL

## Persistence (1/3)



Ensure that from some point on,  $a$  holds but  $b$  does not.

▷ In LTL,  $\Diamond \Box (a \wedge \neg b)$ .

↪ In CTL...?

**This property cannot be expressed in CTL!**

⇒ **Informal argument in the next slide...**



# Formalizing LT/BT properties in CTL

## Persistence (3/3)

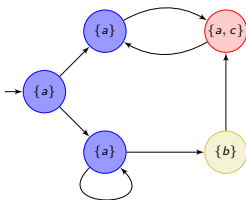
### Intuition.

- In LTL, time is *linear*.
  - ▷ Either we have a path that does branch to  $b$ , thus  $\Box a$  is true after  $b$ . Or we never branch and  $\Box a$  is true from the initial state.
- In CTL, time is *branching*.
  - ▷ We have to use the  $\forall$  quantifier (as we want to characterize all paths).
  - ▷ But then  $\Diamond \forall \Box a$  asks to reach a state where **all possible futures** satisfy  $\Box a$ .
  - ▷ Not possible because of the possibility of branching.

Hence, **even if all branches satisfy  $\Diamond \Box a$** , the CTL formula **requires the additional (and not verified) existence of nodes in the tree whose subtrees only contain paths satisfying  $\Box a$ .**

# Formalizing LT/BT properties in CTL

## Typical BT property



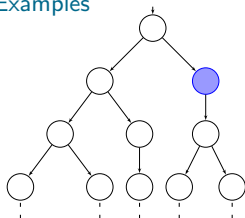
Along all paths, it is always *possible* to reach {a, c}.

- ▷ **Not expressible in LTL**: in linear time, either you reach or you do not. Reasoning about possible futures requires branching time.

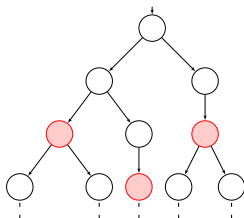
↪ In CTL,  $\forall \square \exists \diamond (a \wedge c)$ .

# CTL semantics

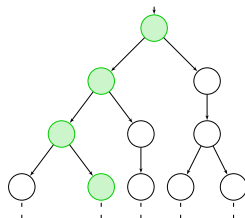
## Examples



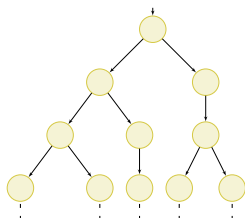
$\exists \bigcirc \text{blue}$



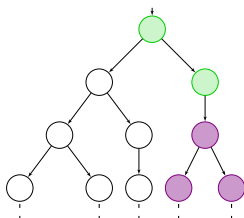
$\forall \Diamond \text{red}$



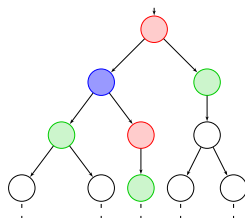
$\exists \Box \text{green}$



$\forall \Box \text{yellow}$



$\exists (\text{green} \cup \forall \Box \text{violet})$



$\forall ((\text{red} \vee \text{blue}) \cup \text{green})$

# CTL semantics

## For state formulae

Let  $\mathcal{T} = (S, Act, \longrightarrow, I, AP, L)$  be a TS without terminal states,  $a \in AP$ ,  $s \in S$ ,  $\Phi$  and  $\Psi$  be CTL state formulae and  $\phi$  be a CTL path formula.

## Satisfaction for state formulae

$s \models \Phi$  iff formula  $\Phi$  holds in state  $s$ .

$$s \models \text{true}$$

$$s \models a \quad \text{iff} \quad a \in L(s)$$

$$s \models \Phi \wedge \Psi \quad \text{iff} \quad s \models \Phi \text{ and } s \models \Psi$$

$$s \models \neg \Phi \quad \text{iff} \quad s \not\models \Phi$$

$$s \models \exists \phi \quad \text{iff} \quad \exists \pi \in Paths(s), \pi \models \phi$$

$$s \models \forall \phi \quad \text{iff} \quad \forall \pi \in Paths(s), \pi \models \phi$$

# CTL semantics

For path formulae

Let  $\pi = s_0 s_1 s_2 \dots$

## Satisfaction for path formulae

$\pi \models \phi$  iff path  $\pi$  satisfies  $\phi$ .

$$\pi \models \bigcirc \Phi \quad \text{iff} \quad s_1 \models \Phi$$

$$\pi \models \Phi \cup \Psi \quad \text{iff} \quad \exists j \geq 0, s_j \models \Psi \text{ and } \forall 0 \leq i < j, s_i \models \Phi$$

$$\pi \models \Diamond \Phi \quad \text{iff} \quad \exists j \geq 0, s_j \models \Phi$$

$$\pi \models \Box \Phi \quad \text{iff} \quad \forall j \geq 0, s_j \models \Phi$$

# CTL semantics

For transition systems

Let  $\mathcal{T} = (S, Act, \longrightarrow, I, AP, L)$  be a TS and  $\Phi$  a CTL state formula over  $AP$ .

## Definition: satisfaction set

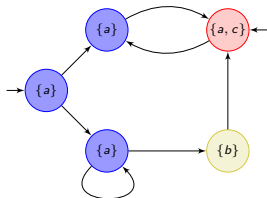
The **satisfaction set**  $Sat_{\mathcal{T}}(\Phi)$  (or briefly,  $Sat(\Phi)$ ) for formula  $\Phi$  is

$$Sat(\Phi) = \{s \in S \mid s \models \Phi\}.$$

TS  $\mathcal{T}$  satisfies  $\Phi$ , denoted  $\mathcal{T} \models \Phi$ , iff  $\Phi$  holds in all initial states, i.e.,

$$\mathcal{T} \models \Phi \text{ iff } I \subseteq Sat(\Phi).$$

# Example



Notice the two initial states.

$$\mathcal{T} \models \forall \bigcirc a$$

$$\mathcal{T} \not\models \exists (a \cup b)$$

$$\mathcal{T} \models \exists \Box a$$

$$\mathcal{T} \models \exists (a \cup c)$$

$$\mathcal{T} \not\models \exists \Diamond b$$

$$\mathcal{T} \not\models \forall \Box a$$

$$\mathcal{T} \models \forall (a \mathcal{W} b)$$

$$\mathcal{T} \models \exists \Box \neg b$$

$$\mathcal{T} \not\models \forall (a \cup b)$$

$$\mathcal{T} \models \forall \Box \exists \Diamond \forall \Box \forall \Diamond c$$

$$\mathcal{T} \models \forall \Box (c \rightarrow \forall \bigcirc a)$$

$$\mathcal{T} \models \exists \Box \exists \Diamond b \rightarrow \neg c$$

$\Rightarrow$  **Blackboard solution.**

# Playing with the semantics

Infinitely often (1/3)

Earlier, we claimed that the CTL formula  $\Phi = \forall \Box \forall \Diamond a$  is *equivalent* to the LTL formula  $\phi = \Box \Diamond a$ , i.e., for all TS  $\mathcal{T}$ ,  $\mathcal{T} \models \Phi$  iff  $\mathcal{T} \models \phi$ .

$\Rightarrow$  **Let's prove it!**

We prove the more precise statement:  $\forall s \in S, s \models \Phi \iff s \models \phi$ , which implies the result for TSs.

# Playing with the semantics

## Infinitely often (2/3)

$$\underline{s \models \Phi \implies s \models \phi.}$$

- 1 Let  $s \models \Phi$ . We must prove that  $\forall \pi = s_0 s_1 s_2 \dots \in Paths(s)$ ,  $\pi \models \phi$ , i.e., for all  $j \geq 0$ , there exists  $i \geq j$  such that  $s_i \models a$ .
- 2 Since  $s \models \forall \Box \forall \Diamond a$  and  $\pi \in Paths(s)$ , we have  $\pi \models \Box \forall \Diamond a$ .
- 3 Hence,  $s_j \models \forall \Diamond a$ .
- 4 Since  $\pi[j..] = s_j s_{j+1} \dots \in Paths(s_j)$ , we have that  $\pi[j..] \models \Diamond a$ .
- 5 Hence, there exists  $i \geq j$  such that  $s_i \models a$ .
- 6 This holds for all  $j$  so we are done.

# Playing with the semantics

Infinitely often (3/3)

$$s \models \Phi \iff s \models \phi.$$

- 1 Let  $s \models \phi$ . We must prove that  $s \models \forall \Box \forall \Diamond a$ , i.e., that  $\forall \pi = s_0 s_1 s_2 \dots \in Paths(s), \pi \models \Box \forall \Diamond a$ .
- 2 I.e., that for all  $j \geq 0$ ,  $s_j \models \forall \Diamond a$ .
- 3 Let  $j \geq 0$  and fix any path  $\pi' = s_j s'_{j+1} s'_{j+2} \dots \in Paths(s_j)$ . We must show that  $\pi' \models \Diamond a$ .
- 4 But, then  $\pi'' = s_0 s_1 \dots s_j s'_{j+1} s'_{j+2} \dots \in Paths(s)$ . Hence,  $\pi'' \models \Box \Diamond a$  by hypothesis.
- 5 Hence, there exists  $i > j$  such that  $s'_i \models a$ .
- 6 Therefore,  $\pi' \models \Diamond a$ .
- 7 This holds for any path  $\pi' \in Paths(s_j)$  so  $s_j \models \forall \Diamond a$ .
- 8 Since it holds for all  $j$ ,  $\pi \models \Box \forall \Diamond a$ .
- 9 Finally, it holds for all  $\pi \in Paths(s)$ , thus  $s \models \Phi$ .

# Semantics of negation

## States

### Negation for states

For  $s \in S$  and a CTL formula  $\Phi$  over  $AP$ ,

$$s \not\models \Phi \iff s \models \neg\Phi.$$

Intuitively, due to the duality between  $\forall$  and  $\exists$  and the semantics of negation for path formulae (see LTL, either a path satisfies  $\phi$  or it satisfies  $\neg\phi$ ).

# Semantics of negation

## Transition systems

### Negation for TSs

For TS  $\mathcal{T} = (S, Act, \longrightarrow, I, AP, L)$  and a CTL formula  $\Phi$  over  $AP$ :

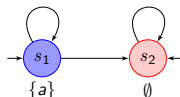
$$\begin{array}{c} \mathcal{T} \not\models \Phi \\ \Downarrow \Uparrow \\ \mathcal{T} \models \neg \Phi \end{array}$$

$$\begin{aligned} \text{We have that } \mathcal{T} \not\models \Phi & \text{ iff } I \not\subseteq Sat(\Phi) \\ & \text{ iff } \exists s \in I, s \not\models \Phi \\ & \text{ iff } \exists s \in I, s \models \neg \Phi \end{aligned}$$

But it may be the case that  $\mathcal{T} \not\models \Phi$  and  $\mathcal{T} \not\models \neg \Phi$  if  
 $\exists s_1, s_2 \in I$  such that  $s_1 \models \Phi$  and  $s_2 \models \neg \Phi$ .

# Semantics of negation

## Example



Consider CTL formula  $\Phi = \exists \Box a$ . Do we have that  $\mathcal{T} \models \Phi$ ?

**Beware of erroneous intuition!**

$$\mathcal{T} \models \exists \phi \not\iff \exists \sigma \in \text{Traces}(\mathcal{T}), \sigma \models \phi.$$

Indeed,  $\Phi$  must hold in **all** initial states.

↪ Here it does not in  $s_2 \implies \mathcal{T} \not\models \Phi$ .

Do we have that  $\mathcal{T} \models \neg \Phi = \forall \Diamond \neg a$ ?

↪ No. Because of path  $(s_1)^\omega$ ,  $s_1 \not\models \neg \Phi \implies \mathcal{T} \not\models \neg \Phi$ .

**Surprising equivalence.**

$$\mathcal{T} \not\models \neg \exists \phi \iff \exists \sigma \in \text{Traces}(\mathcal{T}), \sigma \models \phi.$$

# Equivalence of CTL formulae

## Definition

### Equivalence of CTL formulae

CTL (state) formulae  $\Phi$  and  $\Psi$  over  $AP$  are *equivalent*, denoted  $\Phi \equiv \Psi$ , if and only if, for all TS  $\mathcal{T}$  over  $AP$ ,

$$Sat(\Phi) = Sat(\Psi).$$

In particular,  $\Phi \equiv \Psi \iff (\forall \mathcal{T}, \mathcal{T} \models \Phi \iff \mathcal{T} \models \Psi).$

$\implies$  **Let us review some computational rules.**

# Equivalence of CTL formulae

## Duality for path quantifiers

$$\forall \bigcirc \Phi \equiv \neg \exists \bigcirc \neg \Phi$$

$$\exists \bigcirc \Phi \equiv \neg \forall \bigcirc \neg \Phi$$

$$\forall \Diamond \Phi \equiv \neg \exists \Box \neg \Phi$$

$$\exists \Diamond \Phi \equiv \neg \forall \Box \neg \Phi$$

$$\forall (\Phi \cup \Psi) \equiv \neg \exists (\neg \Psi \cup (\neg \Phi \wedge \neg \Psi)) \wedge \neg \exists \Box \neg \Psi$$

$$\equiv \neg \exists ((\Phi \wedge \neg \Psi) \cup (\neg \Phi \wedge \neg \Psi)) \wedge \neg \exists \Box (\Phi \wedge \neg \Psi)$$

$$\equiv \neg \exists ((\Phi \wedge \neg \Psi) \text{ W } (\neg \Phi \wedge \neg \Psi))$$

$$\exists (\Phi \cup \Psi) \equiv \neg \forall ((\Phi \wedge \neg \Psi) \text{ W } (\neg \Phi \wedge \neg \Psi))$$

# Equivalence of CTL formulae

## Distribution

$$\forall \Box (\Phi \wedge \Psi) \equiv \forall \Box \Phi \wedge \forall \Box \Psi$$

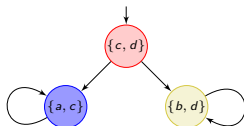
$$\exists \Diamond (\Phi \vee \Psi) \equiv \exists \Diamond \Phi \vee \exists \Diamond \Psi$$

Similar to LTL  $\Box(\phi \wedge \psi) \equiv \Box\phi \wedge \Box\psi$  and  $\Diamond(\phi \vee \psi) \equiv \Diamond\phi \vee \Diamond\psi$ .

**But not all laws can be lifted!**

$$\exists \Box (\Phi \wedge \Psi) \not\equiv \exists \Box \Phi \wedge \exists \Box \Psi$$

$$\forall \Diamond (\Phi \vee \Psi) \not\equiv \forall \Diamond \Phi \vee \forall \Diamond \Psi$$



$$\mathcal{T} \models \forall \Diamond (a \vee b)$$

$$\text{but } \mathcal{T} \not\models \forall \Diamond a \vee \forall \Diamond b$$

$$\mathcal{T} \models \exists \Box c \wedge \exists \Box d$$

$$\text{but } \mathcal{T} \not\models \exists \Box (c \wedge d)$$

# Equivalence of CTL formulae

## Expansion laws

In LTL, we had:

$$\phi \mathbf{U} \psi \equiv \psi \vee (\phi \wedge \bigcirc (\phi \mathbf{U} \psi))$$

$$\diamond \phi \equiv \phi \vee \bigcirc \diamond \phi$$

$$\square \phi \equiv \phi \wedge \bigcirc \square \phi$$

In CTL, we have:

$$\forall (\Phi \mathbf{U} \Psi) \equiv \Psi \vee (\Phi \wedge \forall \bigcirc \forall (\Phi \mathbf{U} \Psi))$$

$$\forall \diamond \Phi \equiv \Phi \vee \forall \bigcirc \forall \diamond \Phi$$

$$\forall \square \Phi \equiv \Phi \wedge \forall \bigcirc \forall \square \Phi$$

$$\exists (\Phi \mathbf{U} \Psi) \equiv \Psi \vee (\Phi \wedge \exists \bigcirc \exists (\Phi \mathbf{U} \Psi))$$

$$\exists \diamond \Phi \equiv \Phi \vee \exists \bigcirc \exists \diamond \Phi$$

$$\exists \square \Phi \equiv \Phi \wedge \exists \bigcirc \exists \square \Phi$$

# Existential normal form (ENF)

## ENF for CTL

### Goal

Retain the full expressiveness of CTL but permit *only existential quantifiers* (thanks to negation and duality).

### ENF for CTL

Given atomic propositions  $AP$ , CTL formulae in *existential normal form* are given by:

$$\Phi ::= \text{true} \mid a \mid \Phi \wedge \Psi \mid \neg\Phi \mid \exists \bigcirc \Phi \mid \exists(\Phi \cup \Psi) \mid \exists \square \Phi$$

where  $a \in AP$ .

**Every CTL formula can be rewritten in ENF... but the translation can cause an exponential blowup (because of the rewrite rule for  $\forall U$ ).**

# Positive normal form (PNF)

## Weak-until PNF for CTL (1/2)

### Goal

Retain the full expressiveness of CTL but permit *only negations of atomic propositions*.

### Weak-until PNF for CTL

Given atomic propositions  $AP$ , CTL state formulae in *weak-until positive normal form* are given by:

$$\Phi ::= \text{true} \mid \text{false} \mid a \mid \neg a \mid \Phi \wedge \Psi \mid \Phi \vee \Psi \mid \exists \phi \mid \forall \phi$$

where  $a \in AP$  and path formulae are given by:

$$\phi ::= \bigcirc \Phi \mid \Phi \cup \Psi \mid \Phi \text{ W } \Psi.$$

# Positive normal form (PNF)

## Weak-until PNF for CTL (2/2)

**Every CTL formula can be rewritten in PNF... but the translation can cause an exponential blowup (because of the rewrite rules for  $\forall U$  and  $\exists U$ ).**

**$\Rightarrow$  As for LTL, can be avoided by introducing a “release” operator.**

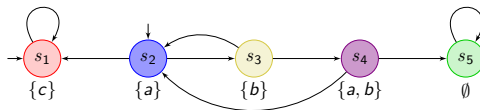
$$\exists(\Phi R \Psi) \equiv \neg \forall((\neg \Phi) U (\neg \Psi))$$

$$\forall(\Phi R \Psi) \equiv \neg \exists((\neg \Phi) U (\neg \Psi))$$

- 1 CTL: a specification language for BT properties
- 2 CTL model checking
- 3 CTL vs. LTL
- 4 CTL\*



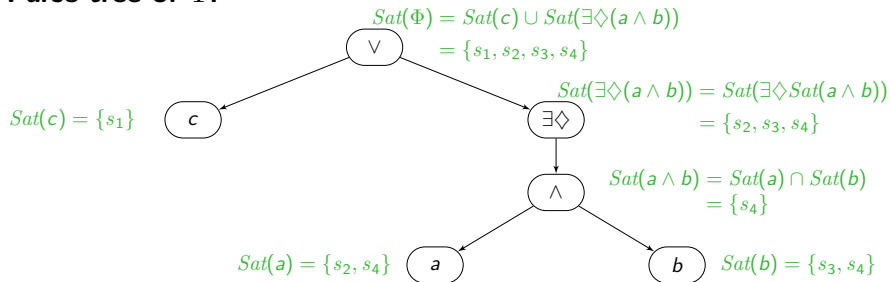
## Toy example



CTL formula  $\Phi = c \vee \exists \Diamond (a \wedge b)$ .

$\Rightarrow$  **We have to check that  $I = \{s_1, s_2\} \subseteq \text{Sat}(\Phi)$ .**

**Parse tree of  $\Phi$ :**



$\Rightarrow$  **Finally  $I \subseteq \text{Sat}(\Phi)$ , thus  $\mathcal{T} \models \Phi$ .**

## Formulae in ENF

Throughout this section, we assume formulae are written in ENF.

### Reminder: ENF for CTL

Given atomic propositions  $AP$ , CTL formulae in *existential normal form* are given by:

$$\Phi ::= \text{true} \mid a \mid \Phi \wedge \Psi \mid \neg\Phi \mid \exists \bigcirc \Phi \mid \exists (\Phi \cup \Psi) \mid \exists \Box \Phi$$

where  $a \in AP$ .

Assume we have  $Sat(\Phi)$  and  $Sat(\Psi)$ , we need algorithms for:

- $Sat(\Phi \wedge \Psi)$  and  $Sat(\neg\Phi)$ : easy, *intersection* and *complement*.
- $Sat(\exists \bigcirc \Phi)$ ,  $Sat(\exists (\Phi \cup \Psi))$  and  $Sat(\exists \Box \Phi)$ .

In practice, one can either rewrite any formula in ENF (but with a potential blow-up), or design specific algorithms to deal with  $\forall$  quantifiers (based on similar ideas).

## Main algorithm

**Key concept:** bottom-up traversal of the parse tree of  $\Phi$ .

For formulae in ENF,

- ▷ leaves can be true or  $a \in AP$ ,
- ▷ inner nodes can be  $\neg$ ,  $\wedge$ ,  $\exists \bigcirc$ ,  $\exists U$ , or  $\exists \square$ .

**Each node represents a subformula  $\Psi$  of  $\Phi$  and  $Sat(\Psi)$  is the set of states where  $\Psi$  holds.**

### Intuition

When we compute  $Sat(\Psi)$  in a node, it is as if we label all states of  $Sat(\Psi)$  with a new proposition  $a_\Psi$  such that  $a_\Psi \in L(s)$  iff  $s \models \Psi$ . This label can then be used to compute the parent formula.

*E.g., computing  $Sat(\exists \bigcirc \Psi)$  is now computing  $Sat(\exists \bigcirc a_\Psi)$ : there is no need to reconsider the child formula  $\Psi$ , just the corresponding labeling of states.*

## Characterization of $Sat$ (1/2)

Let  $\mathcal{T} = (S, Act, \longrightarrow, I, AP, L)$  be a TS without terminal state.  
For all CTL formulae  $\Phi, \Psi$  over  $AP$ , we have:

$$Sat(\text{true}) = S$$

$$Sat(a) = \{s \in S \mid a \in L(s)\} \text{ for } a \in AP$$

$$Sat(\Phi \wedge \Psi) = Sat(\Phi) \cap Sat(\Psi)$$

$$Sat(\neg\Phi) = S \setminus Sat(\Phi)$$

$$Sat(\exists\bigcirc\Phi) = \{s \in S \mid Post(s) \cap Sat(\Phi) \neq \emptyset\}$$

$\hookrightarrow$  All states that have a successor in  $Sat(\Phi)$ .

## Characterization of $Sat$ (2/2)

$Sat(\exists(\Phi \cup \Psi))$  is the smallest subset  $T$  of  $S$  such that

- 1  $Sat(\Psi) \subseteq T$ ,
- 2  $s \in Sat(\Phi) \wedge Post(s) \cap T \neq \emptyset \implies s \in T$ .

$\hookrightarrow$  (1) must hold because  $\Phi \cup \Psi$  is satisfied directly, and (2) says that if  $\Phi$  holds now and there exists a successor where  $\exists(\Phi \cup \Psi)$  holds, then  $\exists(\Phi \cup \Psi)$  holds also now (cf. expansion law).

$Sat(\exists\Box\Phi)$  is the largest subset  $T$  of  $S$  such that

- 1  $T \subseteq Sat(\Phi)$ ,
- 2  $s \in T \implies Post(s) \cap T \neq \emptyset$ .

$\hookrightarrow$  (1) must hold because states outside  $Sat(\Phi)$  directly falsify  $\exists\Box\Phi$ , and (2) says that if  $\exists\Box\Phi$  holds now, then there must exist a successor where  $\exists\Box\Phi$  still holds (cf. expansion law).

# Computation of *Sat*: algorithm (1/3)

**Input:** TS  $\mathcal{T} = (S, Act, \longrightarrow, I, AP, L)$  and CTL formula  $\Phi$  in ENF

**Output:**  $Sat(\Phi) = \{s \in S \mid s \models \Phi\}$

if  $\Phi = \text{true}$  then

return  $S$

else if  $\Phi = a \in AP$  then

return  $\{s \in S \mid a \in L(s)\}$

else if  $\Phi = \Psi_1 \wedge \Psi_2$  then

return  $Sat(\Psi_1) \cap Sat(\Psi_2)$

else if  $\Phi = \neg\Psi$  then

return  $S \setminus Sat(\Psi)$

else if  $\Phi = \exists \bigcirc \Psi$  then

return  $\{s \in S \mid Post(s) \cap Sat(\Psi) \neq \emptyset\}$

⋮

## Computation of *Sat*: algorithm (2/3)

```

:
:
else if  $\Phi = \exists(\Psi_1 \cup \Psi_2)$  then
     $T := \text{Sat}(\Psi_2)$     // smallest fixed point computation
    while  $A := \{s \in \text{Sat}(\Psi_1) \mid \text{Post}(s) \cap T \neq \emptyset\} \neq \emptyset$  do
         $T := T \cup A$ 
    return  $T$ 
:
:

```

$\Leftrightarrow$  We iteratively compute an **increasing** sequence of sets  $T_i$  s.t.  
 $T_0 = \text{Sat}(\Psi_2)$  and  $T_{i+1} = T_i \cup \{s \in \text{Sat}(\Psi_1) \mid \text{Post}(s) \cap T_i \neq \emptyset\}$ ,  
 i.e.,  $T_i$  represents all states that can reach  $\text{Sat}(\Psi_2)$  in at most  $i$   
 steps via a path of states in  $\text{Sat}(\Psi_1)$ .

## Computation of *Sat*: algorithm (3/3)

```

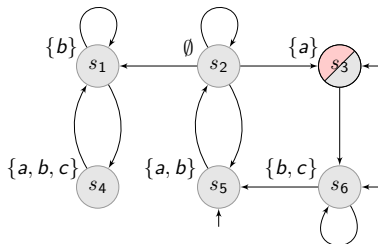
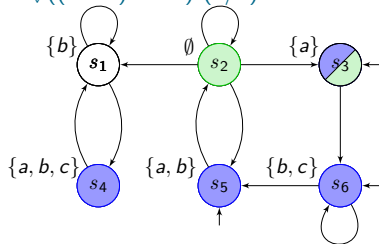
:
:
else if  $\Phi = \exists \square \Psi$  then
     $T := \text{Sat}(\Psi)$  // largest fixed point computation
    while  $A := \{s \in T \mid \text{Post}(s) \cap T = \emptyset\} \neq \emptyset$  do
         $T := T \setminus A$ 
    return  $T$ 

```

$\hookrightarrow$  We iteratively compute a **decreasing** sequence of sets  $T_i$  s.t.  
 $T_0 = \text{Sat}(\Psi)$  and  $T_{i+1} = T_i \cap \{s \in \text{Sat}(\Psi) \mid \text{Post}(s) \cap T_i \neq \emptyset\}$ ,  
i.e.,  $T_i$  represents all states from which there exists a path staying  
in  $\text{Sat}(\Psi)$  for at least  $i$  steps.

# Examples

$$\Phi = \exists \Diamond((a \vee c) \wedge \neg b) \quad (1/2)$$

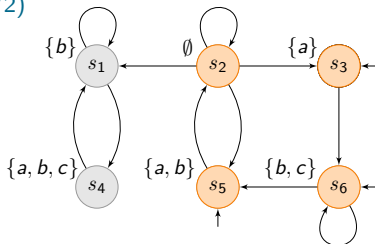


$$\text{Formula } \Phi = \exists \Diamond((a \vee c) \wedge \neg b) \equiv \exists \left( \underbrace{\text{true}}_{\Psi_4} \cup \underbrace{\left( \underbrace{(a \vee c)}_{\Psi_1} \wedge \underbrace{\neg b}_{\Psi_2} \right)}_{\Psi_3} \right)$$

- 1  $Sat(\Psi_1) = Sat(a) \cup Sat(c) = \{s_3, s_4, s_5, s_6\}$
- 2  $Sat(\Psi_2) = S \setminus Sat(b) = \{s_2, s_3\}$
- 3  $Sat(\Psi_3) = Sat(\Psi_1) \cap Sat(\Psi_2), Sat(\Psi_4) = S$
- 4  $Sat(\Phi) = \exists \Psi_4 \cup \Psi_3 \implies$  **Algorithm in the next slide.**

# Examples

$$\Phi = \exists \Diamond((a \vee c) \wedge \neg b) \quad (2/2)$$



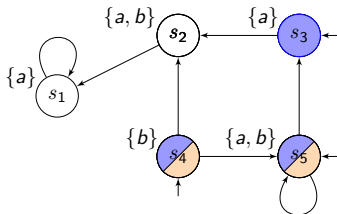
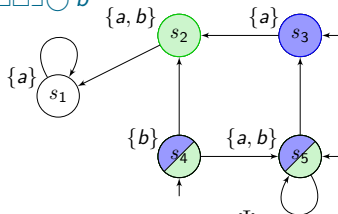
We obtain  $Sat(\Phi) = \exists \Psi_4 \cup \Psi_3$  via smallest fixed point computation:

- ▷  $T_0 = Sat(\Psi_3) = \{s_3\}$
- ▷  $T_1 = T_0 \cup \{s \in Sat(\Psi_4) \mid Post(s) \cap T_0 \neq \emptyset\} = \{s_2, s_3\}$
- ▷  $T_2 = T_1 \cup \{s \in Sat(\Psi_4) \mid Post(s) \cap T_1 \neq \emptyset\} = \{s_2, s_3, s_5\}$
- ▷  $T_3 = T_2 \cup \{s \in Sat(\Psi_4) \mid Post(s) \cap T_2 \neq \emptyset\} = \{s_2, s_3, s_5, s_6\}$
- ▷  $T_4 = T_3 \cup \{s \in Sat(\Psi_4) \mid Post(s) \cap T_3 \neq \emptyset\} = T_3 = Sat(\Phi)$

$$I = \{s_3, s_5, s_6\} \subseteq Sat(\Phi) \implies \mathcal{T} \models \Phi = \exists \Diamond((a \vee c) \wedge \neg b)$$

# Examples

$$\Phi = \exists \square \exists \bigcirc b$$



$$\text{Formula } \Phi = \exists \square \overbrace{\exists \bigcirc b}^{\Psi}$$

- 1  $Sat(b) = \{s_2, s_4, s_5\}$
- 2  $Sat(\Psi) = \{s \in S \mid Post(s) \cap Sat(b) \neq \emptyset\} = \{s_3, s_4, s_5\}$
- 3 We obtain  $Sat(\Phi) = \exists \square \Psi$  via largest fixed point computation:

- ▷  $T_0 = Sat(\Psi) = \{s_3, s_4, s_5\}$
- ▷  $T_1 = T_0 \cap \{s \in Sat(\Psi) \mid Post(s) \cap T_0 \neq \emptyset\} = \{s_4, s_5\}$
- ▷  $T_2 = T_1 \cap \{s \in Sat(\Psi) \mid Post(s) \cap T_1 \neq \emptyset\} = T_1 = Sat(\Phi)$

$$I = \{s_3, s_4, s_5\} \not\subseteq Sat(\Phi) \implies \mathcal{T} \not\models \Phi = \exists \square \exists \bigcirc b$$

- Main algorithm to compute  $Sat(\Phi)$  is a **bottom-up traversal of the parse tree**:  $\mathcal{O}(|\Phi|)$ .

## Complexity of the algorithm

The time complexity is  $\mathcal{O}(|\mathcal{T}| \cdot |\Phi|)$ .

⇒ CTL model checking is in polynomial time!

⇒ So... much more efficient than LTL which is PSPACE-complete?

⇒ **Not really... need to consider the whole picture, including succinctness!**

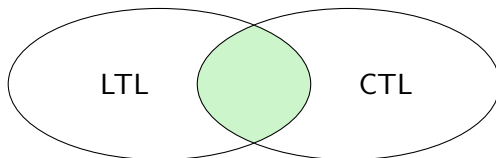
- 1 CTL: a specification language for BT properties
- 2 CTL model checking
- 3 CTL vs. LTL**
- 4 CTL\*

# Expressiveness

## Incomparable logics

We have seen that:

- some properties are **expressible in LTL** but **not in CTL** (e.g.,  $\phi = \Diamond \Box a$ ),
- some properties are **expressible in CTL** but **not in LTL** (e.g.,  $\Phi = \forall \Box \exists \Diamond a$ ),
- some properties **can be expressed in both logics** (e.g.,  $\phi = \Box \Diamond a$  is equivalent to  $\Phi = \forall \Box \forall \Diamond a$ ).



**Can we characterize the intersection?**

# Expressiveness

## Equivalent formulae

Recall the notion of equivalent formulae.

### Definition: equivalent formulae

CTL formula  $\Phi$  and LTL formula  $\phi$  over  $AP$  are *equivalent*, denoted  $\Phi \equiv \phi$  if for all TS  $\mathcal{T}$ ,  $\mathcal{T} \models \Phi \iff \mathcal{T} \models \phi$ .

Here is a way to know if a CTL formula admits an equivalent one in LTL.

### Criterion for transformation from CTL to LTL

Let  $\Phi$  be a CTL formula, and  $\phi$  be the LTL formula obtained by **eliminating all path quantifiers** from  $\Phi$ . Then, either  $\Phi \equiv \phi$  or there exists no LTL formula equivalent to  $\Phi$ .

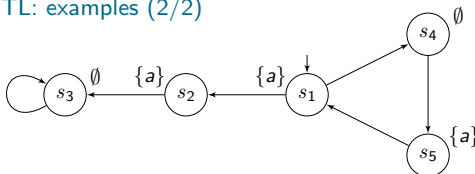
# Expressiveness

## Comparing LTL and CTL: examples (1/2)

- We proved that  $\phi = \Box \Diamond a \equiv \Phi = \forall \Box \forall \Diamond a$ , and indeed,  $\phi$  is obtained from  $\Phi$  by removing all quantifiers.
- We argued that  $\Phi = \forall \Diamond \forall \Box a \not\equiv \phi = \Diamond \Box a$ . Hence, **there is no equivalent to  $\Phi$  in LTL.**

# Expressiveness

## Comparing LTL and CTL: examples (2/2)



Consider formula  $\Phi = \forall \Diamond (a \wedge \forall \bigcirc a)$  and its potential LTL equivalent,  $\phi = \Diamond (a \wedge \bigcirc a)$ .

■  $\mathcal{T} \models \phi$  because  $s_1 \models \phi$ :

- ▷ All paths in  $Paths(s_1)$  contain  $s_1 \rightarrow s_2$ , or  $s_5 \rightarrow s_1$ , or both.
- ▷ Any suffix  $s_1 s_2 \dots$  satisfies  $(a \wedge \bigcirc a)$ , and so does any suffix  $s_5 s_1 \dots$ .
- ▷ Hence all paths satisfy  $\phi$ .

■  $\mathcal{T} \not\models \Phi$  because of path  $s_1 s_2 s_3^\omega$ .

- ▷ None of  $s_1$ ,  $s_2$  and  $s_3$  satisfies  $(a \wedge \forall \bigcirc a)$  (look at  $s_4$  for  $s_1$ ).

$\Rightarrow$  **CTL formula  $\Phi = \forall \Diamond (a \wedge \forall \bigcirc a)$  has no LTL equivalent.**

## Model checking efficiency

Let  $\mathcal{T} = (S, Act, \longrightarrow, I, AP, L)$  be a TS, and  $\Phi$  (resp.  $\phi$ ) a CTL (resp. LTL) formula over  $AP$ .

- Model checking  $\Phi$  requires **linear time in both the model and the formula**:  $\mathcal{O}(|\mathcal{T}| \cdot |\Phi|)$ .
- Model checking  $\phi$  requires **linear time in the model** but **exponential time in the formula**:  $\mathcal{O}(|\mathcal{T}|) \cdot 2^{\mathcal{O}(|\Phi|)}$ .

**Hence, CTL model checking is more efficient, right?**

**No!**

**Because LTL can be exponentially more succinct!**

$\hookrightarrow$  That is, given a CTL formula, the LTL equivalent can be exponentially shorter.

# LTL can be exponentially more succinct than CTL

## Proof sketch (1/3)

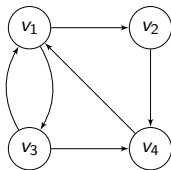
- 1 Take an **NP-complete problem** and show that it can be solved by model checking a **polynomial-size LTL formula on a polynomial-size model**.
- 2 Show that the LTL formula has an **equivalent in CTL (of exponential size)**.
- 3 If an equivalent CTL formula of *polynomial size* existed, **we would be able** to solve the NP-complete problem in polynomial time, hence **to prove that  $P = NP$** .

**Hence, unless  $P = NP$ , some properties can be expressed in LTL through exponentially shorter formulae than in CTL.**

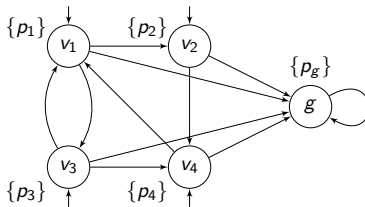
**Chosen problem: deciding the existence of a Hamiltonian path (i.e., visiting each vertex exactly once) in a directed graph.**

# LTL can be exponentially more succinct than CTL

## Proof sketch (2/3)



*Directed graph.*



*Transition system.*

## Encoding of the problem:

- ▶ Make all vertices initial states and add an additional state  $g$  reachable from all other states.
- ▶ Label of vertex  $v_i = p_i$ , label of  $g = p_g$ .
- ▶ Let  $n$  be the number of vertices of the graph. Consider LTL formula  $\phi = (\Diamond p_1 \wedge \dots \wedge \Diamond p_n) \wedge \bigcirc^n p_g$ .
- ▶ *Paths satisfying  $\phi$  in the TS correspond to Hamiltonian paths in the graph.*

# LTL can be exponentially more succinct than CTL

Proof sketch (3/3)

## Reduction:

- ▶ The graph contains a **Hamiltonian path** iff  $\mathcal{T} \not\models \neg\phi$  with  $\phi = (\Diamond p_1 \wedge \dots \wedge \Diamond p_n) \wedge \bigcirc^n p_g$ .
- ▶ Observe that TS  $\mathcal{T}$  and formula  $\phi$  are both of **polynomial size**.
- ▶ No contradiction with NP-completeness since LTL model checking is PSPACE-complete.

## Encoding in CTL?

- ▶ Yes but enumerates all possible Hamiltonian paths! E.g.,

$$\begin{aligned} \Phi = & (p_1 \wedge \exists \bigcirc (p_2 \wedge \exists \bigcirc (p_3 \wedge \exists \bigcirc p_4))) \\ & \vee (p_1 \wedge \exists \bigcirc (p_2 \wedge \exists \bigcirc (p_4 \wedge \exists \bigcirc p_3))) \\ & \vee (p_1 \wedge \exists \bigcirc (p_3 \wedge \exists \bigcirc (p_2 \wedge \exists \bigcirc p_4))) \vee \dots \end{aligned}$$

$\implies$  **Exponential formula:**  $|\Phi| = \mathcal{O}(n \cdot n!)$

$\implies$  **No polynomial encoding can exist unless  $P = NP$**   
**because CTL model checking is in P.**

# Other differences between LTL and CTL

## Fairness

### LTL

- Unconditional, strong and weak **fairness can be formalized in LTL**.
- Fairness can be **incorporated into classical LTL model checking**:  $\mathcal{T} \models_{\text{fair}} \phi$  iff  $\mathcal{T} \models (\text{fair} \rightarrow \phi)$ .

### CTL

- **Most fairness constraints cannot be encoded in CTL**. E.g., strong fairness  $\Box \Diamond a \rightarrow \Box \Diamond b$  is equivalent to  $\Diamond \Box \neg a \vee \Box \Diamond b$  and persistence ( $\Diamond \Box \neg a$ ) is not expressible in CTL.
- Need for  $\forall(\text{fair} \rightarrow \phi)$  and  $\exists(\text{fair} \wedge \phi)$  but **not possible in CTL** (no connectives on path formulae).

$\Rightarrow$  **In CTL, fairness requires specific techniques.**

$\Rightarrow$  Adapt the semantics of  $\exists \phi$  and  $\forall \phi$  to interpret them on **fair** paths, with fairness constraint seen as an LTL formula over CTL state formulae.

$\Rightarrow$  **Not discussed here. See the book for more.**

# Other differences between LTL and CTL

## Implementation relation

### LTL

- LTL is preserved by *trace inclusion* (PSPACE-c.).
- (Bi)simulation is a **sound** but **incomplete** alternative, computable in **polynomial time**.

(bi)simulation



trace inclusion

### CTL

- Bisimulation preserves **full** CTL.
- Simulation preserves **the universal fragment** of CTL.
- ⇨ Allows only quantifier  $\forall$ .
- Equivalently, **simulation preserves the existential fragment** of CTL.
- ⇨ Allows only quantifier  $\exists$  (recall  $\forall\phi \equiv \neg\exists\neg\phi$ ).

⇒ **Different logics, different implementation relations.**

# LTL vs. CTL

## Wrap-up

| <i>Notion of time</i>     | <i>Linear</i>                                                                                     | <i>Branching</i>                                                          |
|---------------------------|---------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------|
| Behavior in state $s$     | path-based: $Traces(s)$                                                                           | state-based:<br>computation tree of $s$                                   |
| Temporal logic            | LTL: path formulae $\phi$<br>$s \models \phi$ iff<br>$\forall \pi \in Paths(s), \pi \models \phi$ | CTL: state formulae $\Phi$<br>path quantifiers $\exists\phi, \forall\phi$ |
| Model checking complexity | PSPACE-complete                                                                                   | P                                                                         |
| Implementation relation   | trace inclusion and equivalence<br>(PSPACE-complete)                                              | (bi)simulation<br>(polynomial time)                                       |

- 1 CTL: a specification language for BT properties
- 2 CTL model checking
- 3 CTL vs. LTL
- 4 CTL\***

## Why?

## Because LTL and CTL are incomparable.

- ▶ CTL\* extends CTL by allowing **arbitrary nesting of path quantifiers** with temporal operators  $\bigcirc$  and  $U$ .
- ▶ **CTL\* subsumes both CTL and LTL.**

⇒ Here, we only take a quick glance at CTL\*. For full discussion, including model checking algorithms, see the book.

# CTL\* syntax

## Core syntax

### CTL\* syntax

Given the set of atomic propositions  $AP$ , CTL\* *state formulae* are formed according to the following grammar:

$$\Phi ::= \text{true} \mid a \mid \Phi \wedge \Psi \mid \neg \Phi \mid \exists \phi$$

where  $a \in AP$  and  $\phi$  is a path formula. CTL\* *path formulae* are formed according to the following grammar:

$$\phi ::= \Phi \mid \phi \wedge \psi \mid \neg \phi \mid \bigcirc \phi \mid \phi \mathbf{U} \psi$$

where  $\Phi$  is a state formula and  $\phi, \psi$  are path formulae.

As for LTL and CTL, we obtain derived propositional logics operators  $\forall, \rightarrow, \dots$ . Moreover,

$$\Diamond \phi = \text{true} \mathbf{U} \phi \quad \text{and} \quad \Box \phi = \neg \Diamond \neg \phi \quad \text{and} \quad \forall \phi = \neg \exists \neg \phi$$

# CTL\* syntax

## Examples (1/2)

### CTL\* syntax reminder

$$\Phi ::= \text{true} \mid a \mid \Phi \wedge \Psi \mid \neg \Phi \mid \exists \phi \quad \phi ::= \Phi \mid \phi \wedge \psi \mid \neg \phi \mid \bigcirc \phi \mid \phi \cup \psi$$

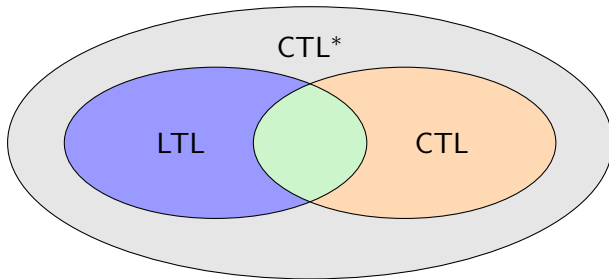
- Is  $\Phi = \exists \bigcirc a$  a valid CTL\* formula? (yes for CTL)
  - ▷ **Yes**, because  $\phi = \bigcirc a$  is a valid path formula, hence  $\Phi = \exists \phi$  is a valid state formula.
- Is  $\Phi = a \wedge b$  a valid CTL\* formula? (yes for CTL)
  - ▷ **Yes**, because  $\Psi_1 = a$  and  $\Psi_2 = b$  are valid state formulae, hence  $\Phi = \Psi_1 \wedge \Psi_2$  is a valid state formula.
- Is  $\Phi = \forall(a \wedge \exists \bigcirc b)$  a valid CTL\* formula? (no for CTL)
  - ▷ **Yes**, because  $\Psi = a \wedge \exists \bigcirc b$  is a valid state formula and any state formula  $\Psi$  can be taken as a path formula  $\phi = \Psi$ .



# Semantics

The semantics of CTL\* follows naturally from the one of CTL.

# Expressiveness



- Any CTL formula is also a CTL\* formula.

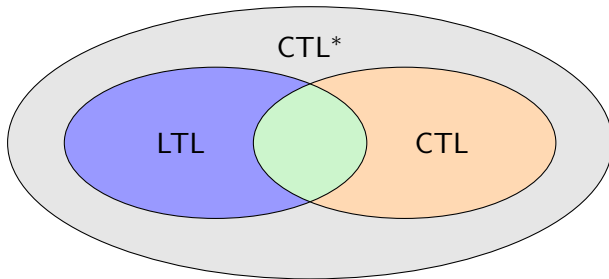
▷ Indeed, the syntax of CTL is a subset of the one of CTL\*.

- Any LTL formula  $\phi$  has an equivalent CTL\* formula.

▷ We have  $\mathcal{T} \models \phi \iff \mathcal{T} \models \Phi = \forall \phi$ .

$\implies$  CTL\* is strictly more expressive than LTL and CTL, i.e., there exist CTL\* formulae that cannot be expressed neither in LTL nor in CTL.

# Expressiveness



Examples of formulae belonging to the different sets

- LTL formula  $\phi = \Diamond \Box a$  cannot be expressed in CTL.
- CTL formula  $\Phi = \forall \Box \exists \Diamond a$  cannot be expressed in LTL.
- LTL formula  $\phi = \Box \Diamond a$  is equivalent to CTL  $\Phi = \forall \Box \forall \Diamond a$ .
- CTL\* formula  $\Phi = \forall \Diamond \Box a \wedge \forall \Box \exists \Diamond b$  is not expressible in LTL nor in CTL.



# Implementation relations

Similarly to CTL,

- bisimulation preserves **full** CTL\*;
- simulation preserves **the existential and universal fragments** of CTL\*.

# References I



C. Baier and J.-P. Katoen.  
*Principles of model checking.*  
MIT Press, 2008.